

Methodology Overview: E-Commerce Conversion Prediction

The Goal The objective was to predict user conversions based on tabular behavioral and demographic data. Since actual conversions are inherently rare in e-commerce, the main challenge was maximizing the F1 Score to ensure a healthy balance between precision and recall.

Data Prep & Clean-up Raw data cannot just be thrown into a model. I handled missing numerical data (like Income and Time on Site) using median values, then scaled them so larger numbers wouldn't unfairly dominate the model. For text-based categorical data (like Device Type or City Tier), I used One-Hot Encoding, specifically configuring it to ignore unknown categories so the pipeline wouldn't break if it encountered new types in the hidden test data.

The Engine: XGBoost

While I established a baseline with a standard Random Forest, I ultimately powered the final submission with XGBoost. It is a massive step up for structured tabular data because it builds decision trees sequentially—meaning each new tree actively learns from and corrects the specific mistakes made by the previous ones.

The Strategy: Optimizing for F1 To get a competitive edge on the F1 metric, I had to handle the massive class imbalance (way more non-buyers than buyers) using two key techniques:

- **Imbalance Weighting:** Instead of letting the model treat both outcomes equally, I calculated the exact ratio of non-conversions to conversions in the training set. I fed this directly into XGBoost, forcing it to pay heavy attention to the rare "Converted" cases.
- **Dynamic Threshold Tuning:** By default, models predict a positive outcome if the probability is 50% or higher. For imbalanced data, 50% is almost never the right cutoff. I shifted the model to predict raw probabilities, then programmatically tested every threshold from 10% to 90% against my validation data to find the exact mathematical sweet spot that yielded the highest F1 Score.

Validation Strategy To ensure the model would actually generalize to unseen data, I trained the XGBoost engine entirely on train.csv. I then used public_test.csv purely as an unbiased validation environment to tune my prediction threshold. Once the optimal threshold was locked in, I applied it to private_test.csv to generate the final binary predictions (0 or 1) for the required submission.